

INFORME FINAL — ERSO 2026-001

- [INFORME FINAL — ERSO 2026-001](#)
 - [Cartera de identidad y custodia de claves en hardware](#)
 - [Cómo leer este informe](#)
 - [Contenido](#)
 - [Resumen ejecutivo](#)
- [PARTE I — MARCO CONCEPTUAL: los términos que necesitas](#)
 - [1. La historia en cinco minutos](#)
 - [2. Diccionario de términos](#)
 - [3. Las siete ideas madre](#)
 - [4. El método de trabajo: cómo se piensa un criterio](#)
- [PARTE II — QUÉ PIDE LA ERSO 001, EXPLICADO FRASE POR FRASE](#)
 - [5. La ficha del documento](#)
 - [6. Capacidades del proceso](#)
 - [7. Condiciones del proceso](#)
 - [8. Descripción del proceso](#)
 - [9. Qué debe hacer: los cinco pasos](#)
 - [10. Los cuatro criterios de aceptación](#)
- [PARTE III — EL PROYECTO: QUÉ SE CONSTRUYÓ PARA ESTA ERSO Y CÓMO ESTÁ MONTADO](#)
 - [11. Visión general](#)
 - [12. El entorno para esta ERSO](#)
 - [13. Mapa de requisitos a implementación](#)
- [PARTE IV — LABORATORIOS DE DEMOSTRACIÓN: LA PRUEBA MATERIAL DE CADA CRITERIO](#)
 - [14. Cómo funcionan los laboratorios](#)
 - [15. Preparación y ejecución del recorrido](#)
 - [16. CRITERIO 1 — La instancia queda registrada y activa, asociada al ciudadano](#)
 - [17. CRITERIO 2 — El par de claves se genera en el hardware seguro y no puede extraerse](#)
 - [18. CRITERIO 3 — El nivel declarado coincide con el realmente disponible](#)
 - [19. CRITERIO 4 — La recuperación de cuenta se completa sin acceder a la clave privada](#)
 - [20. Pruebas automáticas](#)
 - [21. Recorrido de extremo a extremo \(extracto del registro\)](#)
- [PARTE V — PREGUNTAS Y RESPUESTAS](#)
 - [22. Preguntas sobre los conceptos](#)
 - [23. Preguntas por laboratorio y por criterio](#)
- [PARTE VI — CÓMO RESPONDER A CADA CRITERIO](#)
 - [24. Redacción modelo de las cuatro respuestas](#)
 - [25. Lista de comprobación para quien acepta](#)
- [PARTE VII — LÍMITES, HALLAZGOS Y OPERACIÓN](#)
 - [26. Lo que este informe NO demuestra, y lo que se encontró](#)
 - [27. Si algo no sale como en el informe](#)
 - [28. Operación diaria](#)
- [ANEXOS](#)
 - [Anexo A — Los scripts](#)
 - [Anexo B — Autocomprobación \(con respuestas\)](#)
 - [Anexo C — Ejercicios del método \(con respuesta modelo\)](#)

INFORME FINAL — ERSO 2026-001

Cartera de identidad y custodia de claves en hardware

Manual de comprensión, construcción y comprobación

Proyecto	Identidad Digital Soberana SSI
ERSo	2026-001 — Cartera de identidad y custodia de claves en hardware
Desarrollador asignado (según la ERSo)	Luis González
Plantilla de pruebas y pruebas funcionales (según la ERSo)	Geovani Rincón
Fechas de la ERSo	Elaboración: viernes 18-sep-2026 · Desarrollo: lunes 21-sep-2026, 7:30 a. m. · Prueba: miércoles 23-sep-2026, 7:30 a. m.
Dominio del laboratorio	civica-desarrollo.avance.org.co (canal de cartera: puerto 8444)
Ubicación del proyecto	/home/geovani/Descargas/generic/bitacora/new_proyect
Salidas de terminal de este informe	Obtenidas ejecutando los comandos reales sobre el proyecto, el 1 de octubre de 2026
Complementos	scripts/lab-001-completo.sh (todos los laboratorios en un script) · informes/ERSo-2026-001.md (versión corta) · docs/ANDROID-REFERENCIA.md · docs/ANALISIS-BASES-EUDI.md

Nota de fechas. El desarrollo (21-sep) y la prueba (23-sep) que fija la ERSo son anteriores a la fecha de este informe (1-oct-2026). El documento sirve para repetir la prueba o dejar constancia de ella; las firmas de la tabla de actividades son de las personas responsables.

Lo más importante que debes saber antes de empezar. Este proyecto **no tiene un teléfono con hardware seguro real**. El «hardware» es una **simulación en software** (SimulatedHardwareCustodian) que declara honestamente lo que es. Lo que sí se construyó y se probó de verdad es **(a) el contrato** que debe cumplir cualquier hardware seguro y **(b) el servicio del lado del servidor** (Wallet Backend) que **no se deja engañar** sobre el nivel de protección. Cada vez que este informe diga «hardware», léase «hardware simulado», y las secciones de límites (§27) detallan qué falta para llegar a un dispositivo real.

Cómo leer este informe

Símbolo	Significa
	Idea clave: lo más importante de la sección, en una frase

Símbolo	Significa
	Qué vas a ver: lo que debes esperar <i>antes</i> de ejecutar un comando
	Qué significa: cómo interpretar lo que el sistema respondió
	Cuidado: una trampa frecuente o un límite que conviene conocer
	Predice antes de ejecutar: una pregunta para que pienses antes de ver el resultado

Si eres...	Lee esto
Quien decide o aprueba	El Resumen ejecutivo y la Parte VII
Quien necesita entender el problema	Parte I y Parte II
Quien debe reproducir y comprobar	Partes III y IV
Quien prepara la prueba funcional	Partes IV, VI y la lista de comprobación (§25)
Quien prepara una exposición	Partes V y VI

Cómo están presentados los comandos. Los bloques `console` son una **sesión de terminal real**: las líneas con `$` son lo que se escribe; las demás, lo que el sistema respondió. Los valores que cambian en cada ejecución (identificadores, claves, fechas, tokens) serán distintos al repetir; lo que debe coincidir es el **patrón**. Todos los tokens y códigos que aparecen son de **laboratorio y de corta vida**.

Un recurso de este informe: «el recorrido». Para ver paso a paso lo que ocurre entre el dispositivo y el servidor se añadió al simulador del titular (`holder-sim`) un comando de laboratorio, `scenario tour-wallet`, que ejecuta el flujo completo **imprimiendo cada respuesta del servidor**. No cambia nada del producto: solo muestra. Se ejecuta una vez y su salida se explica por secciones.

Contenido

Parte	Secciones	De qué trata
Resumen ejecutivo	—	El problema, la solución y el resultado en una página
I · Marco conceptual	1 a 4	Los términos, en lenguaje sencillo, y el método de trabajo
II · Qué pide la ERSO	5 a 10	Capacidades, condiciones, descripción, pasos y criterios, frase por frase
III · El proyecto	11 a 13	Qué se construyó, cómo se monta en Docker y cómo funciona por dentro
IV · Laboratorios	14 a 21	La prueba material de cada criterio, con cada comando y su respuesta
V · Preguntas y respuestas	22 y 23	Todas las preguntas de comprensión con su respuesta
VI · Cómo responder	24 y 25	Redacción modelo de cada criterio y lista de comprobación
VII · Límites y operación	26 a 28	Lo que NO se demuestra, qué hacer si algo falla y operación diaria

Parte	Secciones	De qué trata
Anexos	A a D	Scripts, autocomprobación, ejercicios y referencias

Resumen ejecutivo

El problema. En una identidad digital soberana, la persona (el *titular*) debe controlar su propia identidad. Eso se reduce a una cosa técnica: **su clave privada nace y vive en su teléfono, dentro de un chip protegido, y nunca sale de él**. Si la clave se copiara a un servidor, la «soberanía» sería una ficción: quien controle el servidor podría actuar en nombre de la persona. Pero aparecen tres dificultades:

1. ¿Cómo sabe el servidor que la clave **realmente** está en un chip protegido y no en la memoria de una aplicación cualquiera (o en un emulador de un atacante)?
2. ¿Qué pasa si el teléfono **se pierde**? El servidor no tiene la clave, así que no puede «devolverla».
3. ¿Cómo se evita que el servidor, que ayuda con la cuenta, **llegue a manejar** la clave?

La solución.

- En el dispositivo: un **contrato** (KeyCustodian) cuyas operaciones **no devuelven jamás una clave privada**; solo permiten *generar, firmar y sellar*.
- En el servidor: un **Wallet Backend** que registra y activa la cartera de cada ciudadano, **exige una prueba (key attestation) firmada por el fabricante y deduce por sí mismo** el nivel de protección en lugar de creerle al dispositivo; rechaza exageraciones, claves importadas y pruebas viejas.
- Recuperación: con un código que se muestra una sola vez, el ciudadano obtiene un token de un solo uso para activar un **dispositivo nuevo con una clave nueva**; el dispositivo anterior queda revocado. **La clave no se recupera: se reemplaza.**

El resultado.

#	Criterio	Resultado	Dónde
1	Instancia registrada y activa, asociada al ciudadano (registro de activación)	✓	§16
2	Par de claves en hardware seguro; la clave privada no se puede exportar	✓ contrato y servidor · ⚠ chip real pendiente	§17
3	Nivel declarado = nivel realmente disponible (ficha técnica y verificación)	✓	§18
4	Recuperación desde el backend sin acceder a la clave privada	✓	§19

Cifras: 37 pruebas automáticas propias de esta ERSO (15 + 7 + 15, todas en verde), un recorrido de laboratorio con 7 intentos de engaño rechazados y una búsqueda de **60 formas textuales de claves privadas** en el volcado completo de la base de datos con **0 coincidencias**.

Lo que debes saber de antemano: (1) el custodio es una simulación; (2) la attestation se verifica contra una autoridad de laboratorio, no contra las raíces reales de Google; (3) la recuperación **no devuelve la clave**: las credenciales deben reemitirse y el DID publicado con la clave perdida queda huérfano (§26).

PARTE I — MARCO CONCEPTUAL: los términos que necesitas

1. La historia en cinco minutos

Imagina una **caja fuerte portátil** que le entregan a cada ciudadano. La caja tiene una ranura: se le pueden **pasar documentos para que los firme por dentro**, y salen firmados; pero **nadie puede sacar la llave**, ni siquiera su dueño, ni siquiera quien la fabricó. Esa caja es el **hardware seguro** del teléfono.

- Cuando el ciudadano «abre una cuenta», su teléfono **crea una llave nueva dentro de la caja** (no la trae de fuera).
- Para que el **servidor** (el Wallet Backend) confíe, no le basta con que el teléfono diga «está en una caja fuerte». Le pide un **certificado de fábrica** —firmado por el fabricante— que dice: «esta llave nació dentro de mi caja de nivel X, y el teléfono arrancó con software legítimo».
- El servidor **verifica el certificado** y **anota el nivel que él mismo comprobó**, no el que el teléfono afirma.
- Si el teléfono se pierde, el ciudadano usa un **código secreto de recuperación** para activar un teléfono nuevo con una llave nueva. La vieja queda anulada. El servidor **nunca tuvo** la llave; por eso tampoco puede devolverla.

💡 Esta ERSO trata de que **el control de la identidad quede en el dispositivo del ciudadano**, y de que el servidor **coordine sin poder usar** esa llave.

2. Diccionario de términos

2.1 Claves y custodia

Término	En palabras sencillas
Titular / ciudadano / holder	La persona dueña de la identidad y de la cartera.
Cartera (wallet)	La aplicación del ciudadano donde viven sus claves, su DID y sus credenciales.
Instancia de cartera	Una instalación concreta de la cartera en un dispositivo concreto. Un ciudadano tiene una activa a la vez.
Clave privada	El «sello secreto»: firma y nunca se comparte .
Clave pública	La contraparte: verifica las firmas y puede ser pública .
P-256 / ES256	El tipo de clave y el algoritmo de firma que usa el proyecto.
Huella (thumbprint, RFC 7638)	Un resumen de la clave pública que sirve como identificador estable.

Término	En palabras sencillas
Custodio de claves (KeyCustodian)	La pieza de software que guarda las claves y firma por ellas . Es la abstracción del hardware seguro.
No exportable	Que la clave puede usarse (firmar) pero no leerse ni copiarse .
Generada en el dispositivo (<i>origin</i> = GENERATED)	La clave nació dentro del custodio. Una clave importada de fuera no puede afirmar ser no exportable: alguien ya la vio.
Sellar	Cifrar datos locales con una clave interna del custodio que tampoco sale.

2.2 Niveles de protección

Término	En palabras sencillas
SOFTWARE	La clave vive en la memoria de la aplicación. Sin protección de hardware.
TEE (entorno de ejecución de confianza)	Una zona aislada del procesador principal donde se hacen las operaciones sensibles.
STRONGBOX	Un chip de seguridad dedicado , aparte del procesador: el nivel más alto en Android.
Android Keystore / StrongBox	El servicio de Android que guarda claves en TEE o StrongBox.
iOS Keychain / Secure Enclave	Los equivalentes en iPhone.
Pedir ≠ tener	Pedirle al sistema «usa StrongBox» es una petición ; si el aparato no lo tiene, puede caer a TEE o a software sin avisar . Por eso hay que <i>declarar el nivel realmente disponible</i> .
Política de nivel mínimo	Regla que dice «no acepto carteras por debajo de este nivel».

2.3 Pruebas del dispositivo

Término	En palabras sencillas
Key attestation	Un certificado que el hardware mismo produce al crear una clave, firmado por el fabricante, diciendo qué nivel de seguridad tiene la clave, de dónde vino y cómo arrancó el aparato.
Cadena de certificados	Una lista de certificados encadenados desde el del dispositivo hasta una raíz conocida.
Raíz de confianza (ancla)	El certificado en el que el servidor confía de antemano (en producción: las raíces de Google). Si la cadena no llega a una raíz conocida, se rechaza.
Arranque verificado (verified boot)	El aparato comprobó que su sistema operativo no fue alterado. Si está desbloqueado, no se puede confiar en el hardware.
Desafío (challenge, nonce)	Un valor aleatorio de un solo uso que el servidor entrega; el dispositivo lo incrusta en la attestation. Así una attestation vieja no sirve.
Prueba de posesión	Firma del desafío con la clave que se declara: demuestra que quien activa controla esa clave.
JWS / JWT	Formato de texto firmado: <code>cabecera.contenido.firma</code> .

Término	En palabras sencillas
Autoridad de laboratorio	Una autoridad simulada que firma las attestations de prueba con la misma estructura que las reales.

2.4 Servidor y recuperación

Término	En palabras sencillas
Wallet Backend	El servicio del servidor que registra carteras, verifica attestation y coordina la recuperación. Nunca ve una clave privada.
Activar	Dejar una instancia lista y asociada al ciudadano, tras verificarla.
Revocar	Anular una instancia: deja de autenticar.
Registro de activación	La anotación en la auditoría de que se activó una instancia.
Auditoría inalterable (<i>append-only</i>)	Registro donde solo se puede agregar; la base de datos rechaza modificar o borrar.
Índice único parcial	Regla de la base de datos: «solo una fila <code>ACTIVE</code> por ciudadano».
Código de recuperación	Secreto que se muestra una sola vez al crear la cuenta. El servidor guarda solo un resumen (PBKDF2 con sal), no el código.
PBKDF2	Forma deliberadamente lenta de resumir un secreto, para dificultar adivinarlo por fuerza bruta.
Token de recuperación	Un pase de un solo uso que se obtiene con el código correcto y permite activar un dispositivo nuevo.
Bloqueo por intentos	Tras 5 intentos fallidos, se bloquea la recuperación 15 minutos.
Reemisión	Volver a emitir las credenciales, porque estaban ligadas a la clave anterior.

3. Las siete ideas madre

1. **La clave nace y vive en el dispositivo.** El servidor no la tiene ni la puede pedir.
2. **Una clave no exportable se puede usar, no leer.** Es la única garantía que importa.
3. **No se le cree al dispositivo: se le exige prueba.** La attestation firmada por el fabricante.
4. **El servidor deduce el nivel; no lo acepta.** Declarar más, o menos, se rechaza.
5. **Una prueba vieja no sirve.** Desafío de un solo uso.
6. **Una cartera activa por ciudadano,** garantizado por la base de datos.
7. **Recuperar es reemplazar, no restaurar.** Clave nueva, cartera nueva, la vieja revocada.

4. El método de trabajo: cómo se piensa un criterio

1. **Descomponer** el criterio en el verbo y la evidencia que pide.
2. **Hacerlo comprobable:** ¿qué comando lo demuestra?
3. **Buscar el caso negativo:** lo que **debe** fallar.
4. **Predecir** el resultado.
5. **Ejecutar y observar.**
6. **Correlacionar** con la regla del código.

7. **Redactar** la respuesta con la evidencia **y con el límite** si lo hay.

PARTE II — QUÉ PIDE LA ERSo 001, EXPLICADO FRASE POR FRASE

5. La ficha del documento

Campo del PDF	Valor
Proyecto	Identidad Digital Soberana SSI
Título	Cartera de identidad y custodia de claves en hardware
Desarrollador	Luis González
Fecha de la ERSo	Viernes, 18 de septiembre de 2026
Fecha de desarrollo	Lunes, 21 de septiembre de 2026, 7:30 a. m.
Fecha de prueba	Miércoles, 23 de septiembre de 2026, 7:30 a. m.
Responsables	Análisis y diseño, asignar desarrollador, aceptar archivos y archivo: Rocío Villamizar · Implementar: Luis González · Plantilla de pruebas y pruebas funcionales: Geovani Rincón

⚠ Las firmas y fechas de la tabla de actividades **corresponden a las personas responsables**; este informe aporta las **evidencias técnicas**, no las firmas.

Capacidades → Condiciones → Descripción → Qué debe hacer (5 pasos) → Criterios (4) → Actividades

6. Capacidades del proceso

6.1 La frase principal, desarmada

"Se requiere habilitar una **cartera de identidad bajo control del ciudadano**, con **generación y custodia local de claves privadas en el hardware seguro** del dispositivo, y con capacidad de **gestionar instancias, recuperación de cuenta y servicios auxiliares desde el backend** de la cartera."

#	Fragmento	En lenguaje sencillo	Cómo se materializó
1	"bajo control del ciudadano"	Quien manda es la persona, no el servidor	La clave nace y vive en el dispositivo
2	"generación y custodia local"	La clave se crea y se guarda en el aparato	KeyCustodian.generate

#	Fragmento	En lenguaje sencillo	Cómo se materializó
3	"hardware seguro"	En un chip protegido, no en memoria común	Contrato + attestation (simulado en el laboratorio)
4	"gestionar instancias"	Registrar y activar cada instalación	POST /wallet/v1/instances
5	"recuperación de cuenta"	Reponerse de un teléfono perdido	Código → token → dispositivo nuevo
6	"servicios auxiliares desde el backend"	El servidor ayuda, pero no custodia	Auditoría, respaldo cifrado opaco, publicación del DID

6.2 Las cinco capacidades

#	Capacidad (texto del PDF)	Qué se busca	Cómo se resolvió	Paso
1	Crear y administrar la instancia de cartera del ciudadano	Un registro de cada cartera, ligado a su dueño	wallet_instances + activación	1
2	Generar pares de claves privadas dentro del entorno protegido del dispositivo	Que la clave nazca dentro	KeyCustodian.generate	2
3	Custodiar las claves privadas de modo no exportable	Que no se pueda sacar	Contrato sin métodos que devuelvan la clave	3
4	Declarar el nivel de protección realmente disponible	No prometer más de lo que hay	ProtectionLevel + verificación por attestation	4
5	Coordinar desde el backend la recuperación de cuenta y los servicios auxiliares, sin comprometer la clave privada	Reponerse sin que el servidor vea claves	Código de recuperación + token	5

7. Condiciones del proceso

Condición 1 — "Las claves privadas son no exportables y nunca salen del dispositivo."

- **Qué significa.** La clave se puede usar para firmar, pero no leer ni copiar.
- **Qué problema evita.** Que alguien (un virus, un administrador, un robo de copia de seguridad) se lleve la clave y suplante a la persona.
- **Cómo se cumple.** El contrato KeyCustodian no tiene ninguna operación que devuelva una clave privada; el servidor ni siquiera tiene una columna donde guardarla (§17).

Condición 2 — "El backend no accede a la clave privada del ciudadano."

- **Qué significa.** El servidor coordina, no custodia.
- **Qué problema evita.** Que un fallo o abuso del servidor comprometa a todos los ciudadanos a la vez.
- **Cómo se cumple.** Ninguna API recibe una clave privada; una búsqueda de las claves reales en **toda** la base de datos da cero resultados (§17.3).

Condición 3 — "Se declara explícitamente el nivel de protección realmente disponible en el dispositivo."

- **Qué significa.** Si el aparato solo tiene protección de software, así se dice; no se «infla».
- **Qué problema evita.** Falsa seguridad: tratar como blindada una clave que está en memoria común.
- **Cómo se cumple.** El servidor **deduce** el nivel a partir de la attestation y lo compara con lo declarado; si no coinciden, rechaza (§18).

8. Descripción del proceso

"La cartera de identidad es el componente del ciudadano donde nace, vive y se usa su clave privada: debe generarse y firmar dentro del hardware seguro del dispositivo y no copiarse al servidor."

Tres verbos: **nace, vive, se usa** —todo dentro del dispositivo—.

"El objetivo es que la custodia de la clave quede del lado del usuario y que el backend solo coordine servicios auxiliares, sin poder usar ni extraer esa clave."

Dos prohibiciones al servidor: **no usar** y **no extraer**.

"Literatura y temas a consultar"

Lectura	Qué te aporta
Android Keystore y StrongBox	Cómo se generan y custodian claves en Android, y los niveles TEE/StrongBox
iOS Secure Enclave	El equivalente en iPhone
WebAuthn y FIDO2	La idea de autenticación ligada al dispositivo con clave respaldada por hardware
Modelo de datos de Credenciales Verificables de W3C	Qué es el <i>holder</i> (titular)

Enlaces en [docs/VERSIONES-NORMATIVAS.md](#) y análisis de Android en [docs/ANDROID-REFERENCIA.md](#).

9. Qué debe hacer: los cinco pasos

1. Construir el servicio que registra y activa la instancia de cartera de cada instalación, asociada al ciudadano.
2. Implementar la generación del par de claves dentro del entorno protegido del dispositivo (Keystore/Keychain o equivalente), sin exportar la clave privada.
3. Dejar la clave privada en custodia no exportable, con la marca de no exportabilidad activada.

4. Detectar y declarar el nivel de protección realmente disponible en el dispositivo, sin asumir garantías superiores a las que el hardware ofrece.
5. Implementar desde el backend la coordinación de la recuperación de cuenta y los servicios auxiliares, sin acceso a la clave privada.

Paso	Lo que pide	Criterio que lo comprueba	Laboratorio
1	Servicio que registra y activa	Criterio 1	\$16
2	Generar la clave dentro del entorno protegido	Criterio 2	\$17
3	Custodia no exportable con su marca	Criterio 2	\$17
4	Detectar y declarar el nivel real	Criterio 3	\$18
5	Recuperación sin acceder a la clave	Criterio 4	\$19

10. Los cuatro criterios de aceptación

1. La instancia de cartera queda registrada y activa, asociada al ciudadano; evidencia: registro de activación. **2.** El par de claves se genera dentro del hardware seguro y la clave privada no puede extraerse ni exportarse; evidencia: prueba de no exportabilidad. **3.** El nivel de protección declarado coincide con el realmente disponible en el dispositivo; evidencia: ficha técnica y verificación del dispositivo. **4.** La recuperación de cuenta se completa desde el backend sin acceder a la clave privada; evidencia: prueba de recuperación.

Criterio	Verbo y evidencia	Lo que debo poder mostrar	El caso negativo	Mecanismo
1	Queda registrada y activa · registro de activación	Una activación con estado <code>ACTIVE</code> y una entrada de auditoría	Una segunda cartera activa; desafío reutilizado	Índice único parcial + auditoría
2	No puede extraerse · prueba de no exportabilidad	El contrato no ofrece forma de leer la clave; el servidor no la guarda; clave importada rechazada	Clave importada; búsqueda en la base	Contrato + <code>origin=GENERATED</code>
3	Coincide · ficha técnica y verificación	Ficha con nivel declarado = verificado	Exagerar, declarar de menos, raíz falsa, attestation vieja, arranque no verificado	<code>KeyAttestationVerifier</code>
4	Se completa sin acceder a la clave · prueba de recuperación	Recuperación completa con dispositivo nuevo	Código erróneo, token reutilizado, fuerza bruta	Código + token de un solo uso + bloqueo

⚠ Sobre el criterio 2. Dice «hardware seguro». Con un dispositivo simulado se puede probar el contrato y el comportamiento del servidor, pero **no** que un chip real se niegue a exportar la clave. Esa parte requiere un teléfono y se deja como plantilla de prueba (§26).

PARTE III — EL PROYECTO: QUÉ SE CONSTRUYÓ PARA ESTA ERSO Y CÓMO ESTÁ MONTADO

11. Visión general

11.1 Qué se construyó

Pieza	Dónde corre	Qué hace
Wallet Backend (<code>wallet-service</code>)	Contenedor <code>wallet</code>	Registra y activa carteras, verifica attestation, coordina recuperación, respalda
Contrato <code>KeyCustodian</code> (<code>wallet-core</code>)	Biblioteca (en el dispositivo)	Define qué puede hacer el hardware seguro: generar, firmar, sellar. Nunca devolver una clave privada
<code>SoftwareKeyCustodian</code>	Biblioteca	Custodio de software que declara <code>SOFTWARE</code>
<code>SimulatedHardwareCustodian</code>	Biblioteca de laboratorio	Simula TEE/StrongBox con attestation firmada por la autoridad de laboratorio
<code>LabAttestationAuthority</code>	Biblioteca de laboratorio	Hace de «fabricante»: firma attestations con la estructura real de Android
<code>KeyAttestationVerifier</code>	En el backend	Lee la attestation y deduce nivel, origen y arranque
Holder App simulada (<code>holder-sim</code>)	Contenedor <code>holder</code> (perfil <code>tools</code>)	Hace de teléfono: ejecuta el flujo real contra el servidor por la red
Tablas <code>wallet_*</code>	Contenedor <code>postgres</code>	Ciudadanos, instancias, desafíos, tokens, auditoría

11.2 Las piezas en Docker

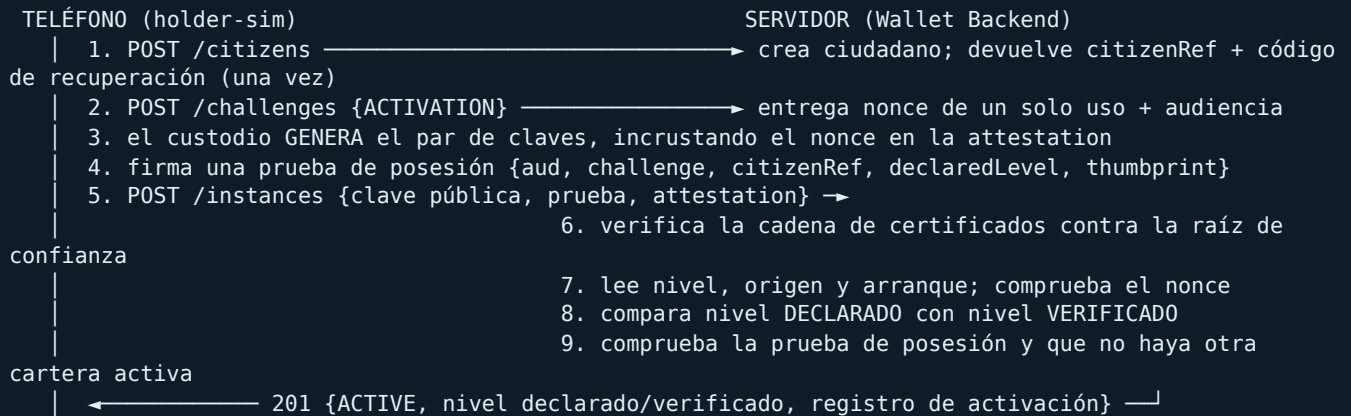
Contenedor	Para qué sirve en esta ERSO
<code>nginx</code>	Puerta <code>8444</code> (canal de cartera y credenciales), con TLS
<code>wallet</code>	El Wallet Backend (puerto interno <code>8090</code> ; no se publica al exterior)
<code>postgres</code>	Base de datos con las tablas <code>wallet_*</code>
<code>vdr</code> , <code>credential</code>	Otros servicios; no intervienen aquí (salvo <code>vdr</code> para publicar el DID, ERSO <code>003</code>)
<code>holder</code> (perfil <code>tools</code>)	El teléfono simulado, bajo demanda

```
$ docker compose ps --format 'table {{.Service}}\t{{.Status}}\t{{.Ports}}'
SERVICE      STATUS      PORTS
credential     Up 3 minutes
nginx          Up 3 minutes      80/tcp, 0.0.0.0:8444->8444/tcp, [::]:8444->8444/tcp,
0.0.0.0:8443->443/tcp, [::]:8443->443/tcp, 0.0.0.0:9443->8443/tcp, [::]:9443->8443/tcp
```

```
postgres    Up 3 minutes (healthy)   5432/tcp
vdr         Up 3 minutes
wallet      Up 3 minutes
```

✓ Solo `nginx` publica puertos al equipo. `wallet`, `credential`, `vdr` y `postgres` **no** tienen puertos hacia fuera: solo se alcanzan por la red interna de Docker.

11.3 El recorrido de una activación



💡 Los pasos 6 a 9 son las **puertas** del servidor. Cualquiera que falle devuelve un error con su motivo, y la activación no ocurre.

12. El entorno para esta ERSO

12.1 Preparar el terminal

Atajo	Qué hace
H	Ejecuta el teléfono simulado (<code>holder</code>) en su contenedor
WALLET <escenario>	Ejecuta un escenario del teléfono contra el servidor real (canal 8444 , CA de laboratorio, autoridad de attestation de laboratorio)
PSQL	Entra a la base de datos
CW	Opciones de <code>curl</code> hacia el canal de cartera (8444)

```
$ cd /home/geovani/Descargas/generic/bitacora/new_proyect/deploy
$ set -a; . ./env; set +a; export LOCAL_UID=$(id -u) LOCAL_GID=$(id -g)
$ D=$VDR_DOMAIN
$ W=../evidencias/work
$ BASE=https://$D:8444
$ PUB="--cacert certs/ca.crt --resolve $D:8443:127.0.0.1"
$ CW="--cacert certs/ca.crt --resolve $D:8444:127.0.0.1"
$ PSQL="docker compose exec -T postgres psql -U $POSTGRES_USER -d $POSTGRES_DB"
$ H() { docker compose --progress quiet --profile tools run --rm -T holder "$@"; }
$ WALLET() { H escenario "$1" --authority authority.json --domain $D --base $BASE --ca /certs/ca.crt "${@:2}"; }
$ CRED() { H escenario "$1" --authority authority.json --domain $D --base $BASE --ca /certs/ca.crt --admin-url https://$D:8443 --p12 /certs/vdr-admin.p12 --p12-pass $P12_PASS --admin-token $CREDENTIAL_ADMIN_TOKEN "${@:2}"; }
```

12.2 El canal de cartera, visto desde fuera

```
$ curl -s -o /dev/null -w "wallet/v1 sin token por el canal 8444 -> HTTP %{http_code}\n" $CW
$BASE/wallet/v1/instances/00000000-0000-0000-0000-000000000000
wallet/v1 sin token por el canal 8444 -> HTTP 401
```

- **Qué hace.** Pide un recurso de cartera **sin token**.
- **Qué significa.** `401`: el servidor responde y protege lo que debe proteger. (El alta del ciudadano y los desafíos son abiertos en el laboratorio; ver §26.)

12.3 La puerta de nginx para la cartera

```
$ sed -n '/listen 8444/,/location \/ { return 404; }/p' nginx/nginx.conf
listen 8444 ssl;
server_name civica-desarrollo.avance.org.co localhost;
ssl_certificate /etc/nginx/certs/server.crt;
ssl_certificate_key /etc/nginx/certs/server.key;

add_header X-Content-Type-Options nosniff always;
client_max_body_size 256k;

# API del Wallet Backend (gestión de cartera y del DID del titular)
location /wallet/v1/ { set $wallet_upstream http://wallet:8090; proxy_pass $wallet_upstream;
}

# OpenID4VCI 1.0 (emisión) y OpenID4VP 1.0 (presentación): los únicos endpoints de
credenciales
location = /.well-known/openid-credential-issuer { limit_except GET { deny all; } set
$credential_upstream http://credential:8100; proxy_pass $credential_upstream; }
location = /.well-known/oauth-authorization-server { limit_except GET { deny all; } set
$credential_upstream http://credential:8100; proxy_pass $credential_upstream; }
location = /token { limit_except POST { deny all; } set
$credential_upstream http://credential:8100; proxy_pass $credential_upstream; }
location = /nonce { limit_except POST { deny all; } set
$credential_upstream http://credential:8100; proxy_pass $credential_upstream; }
location = /credential { limit_except POST { deny all; } set
$credential_upstream http://credential:8100; proxy_pass $credential_upstream; }
location = /verifier/response { limit_except POST { deny all; } set
$credential_upstream http://credential:8100; proxy_pass $credential_upstream; }

location / { return 404; }
```

- `listen 8444 ssl` — TLS con el certificado del servidor.
- `location /wallet/v1/ { ... proxy_pass http://wallet:8090; }` — la API del Wallet Backend.
- Las demás rutas son los únicos endpoints de OpenID4VCI/OpenID4VP (ERSo 002); cualquier otra ruta devuelve `404`.

12.4 Las tablas del servidor

```
$ echo "\dt wallet*" | $PSQL
List of relations
Schema | Name | Type | Owner
-----+-----+-----+-----
public | wallet_audit | table | vdr
public | wallet_challenges | table | vdr
public | wallet_citizens | table | vdr
public | wallet_did_backups | table | vdr
public | wallet_did_publications | table | vdr
public | wallet_instances | table | vdr
```

```
public | wallet_recovery_tokens | table | vdr
(7 rows)
```

Tabla	Guarda
wallet_citizens	El ciudadano (id opaco), sal y resumen del código de recuperación, intentos fallidos y bloqueo
wallet_instances	Cada cartera: estado, clave pública , huella, nivel declarado y verificado, ficha del dispositivo, resumen del token
wallet_challenges	Desafíos de un solo uso
wallet_recovery_tokens	Tokens de recuperación (solo su resumen)
wallet_audit	Auditoría inalterable
wallet_did_publications , wallet_did_backups	Publicación y respaldo del DID (ERSo 003)

⚠ Observa lo que **no** hay: ninguna tabla tiene una columna de clave privada.

12.5 La configuración de seguridad

```
$ grep -nE 'WALLET_(MIN_LEVEL|REQUIRE_VERIFIED_BOOT|ATTESTATION_ROOTS)|recovery|maxAttempts|lock'
../wallet-service/src/main/kotlin/co/org/avance/ssi/wallet/Config.kt | cut -c1-170
17:   val recoveryTokenTtlSeconds: Long = 600,
19:   val recoveryLockMinutes: Long = 15,
40:       minimumLevel = env["WALLET_MIN_LEVEL"]?.let { ProtectionLevel.valueOf(it) } ?:
ProtectionLevel.SOFTWARE,
41:       requireVerifiedBoot = env["WALLET_REQUIRE_VERIFIED_BOOT"]?.toBooleanStrictOrNull() ?:
true,
42:       attestationRootsPem = env["WALLET_ATTESTATION_ROOTS"]?.takeIf { it.isNotBlank() &&
java.io.File(it).exists() }?.let { java.io.File(it).readText() },
```

- `WALLET_MIN_LEVEL` — nivel mínimo aceptado (en laboratorio `SOFTWARE` para poder probar; en producción debería ser `TEE` o superior).
- `WALLET_REQUIRE_VERIFIED_BOOT` — exige arranque verificado (por defecto sí).
- `WALLET_ATTESTATION_ROOTS` — las raíces de confianza (en laboratorio, la de la autoridad simulada).
- Recuperación: token de **600 s** y bloqueo de **15 minutos**.

12.6 Cómo se construye el teléfono simulado

El contenedor `holder` no compila nada dentro: el binario se construye en el equipo (`./gradlew :wallet-sim:installDist`) y se copia a la imagen (`docker compose --profile tools build holder`).

Si cambias el código del simulador hay que repetir ambos pasos.

13. Mapa de requisitos a implementación

Paso	Implementación	Dónde
1 · Registrar y activar	<code>POST /wallet/v1/instances</code> ; índice único parcial; auditoría	<code>WalletService.kt</code> , <code>W1__wallet.sql</code>

Paso	Implementación	Dónde
2 · Generar en el entorno protegido	KeyCustodian.generate	wallet-core/.../KeyCustodian.kt , SoftwareKeyCustodian.kt , SimulatedHardwareCustodian
3 · No exportable, con marca	KeyDescriptor.exportable=false , origin=GENERATED ; rechazo si origin ≠ GENERATED	KeyDescriptor , KeyAttestationVerifier
4 · Declarar el nivel real	ProtectionLevel , KeyPolicy ; verificación y comparación estricta en el backend	KeyAttestation.kt , WalletService.evaluate/activate
5 · Recuperación	/recovery/start → token → activación que revoca la anterior	WalletService.recoveryStart

PARTE IV — LABORATORIOS DE DEMOSTRACIÓN: LA PRUEBA MATERIAL DE CADA CRITERIO

14. Cómo funcionan los laboratorios

Cada laboratorio:  **predicción** → **comandos** (con su respuesta real y su explicación) →  **conclusión**.

El laboratorio central es **el recorrido** (`WALLET tour-wallet`): se ejecuta **una sola vez** y su salida se reparte entre los laboratorios. Verás el mismo comando al comienzo de cada bloque, seguido de un comentario `# ...` (extracto...) que indica qué sección del recorrido se muestra.

Lab	Criterio	Qué se prueba	Sección
A	1	Activar y registrar la instancia	§16
B	2	No exportabilidad	§17
C	3	El nivel declarado = el real	§18
D	4	Recuperación de cuenta	§19
E	—	Pruebas automáticas	§20
F	—	Recorrido de extremo a extremo	§21

15. Preparación y ejecución del recorrido

Preparación: §12.1. El recorrido se ejecuta en el laboratorio A; su salida completa se reparte entre las secciones §16 a §19.

16. CRITERIO 1 — La instancia queda registrada y activa, asociada al ciudadano

Criterio. "La instancia de cartera queda registrada y activa, asociada al ciudadano; evidencia: registro de activación."

Predice antes de ejecutar

(1) Al activar, el servidor devuelve un estado, un token y un «registro de activación». ¿Qué crees que contiene este último? (2) Si el mismo ciudadano intenta activar una **segunda** cartera, ¿qué ocurre y quién lo impide: la aplicación o la base de datos?

16.1 El alta del ciudadano

```
$ OUT=$(WALLET tour-wallet --secrets-out secrets-wallet.txt); echo "$OUT"
# ... (extracto del recorrido: sección 1; el resto del recorrido se muestra en las demás secciones)
— 1. Alta del ciudadano (el backend entrega un identificador opaco y un código de recuperación, una sola vez)
  citizenRef          : b49dbd5e-4ae4-4ea5-98cf-6b01db443cce
  recoveryCode        : NE96-HD32-LTZA-8EG5-G3W3
```

- **Qué hace.** POST `/citizens` crea un ciudadano y devuelve un **identificador opaco** (`citizenRef`) y un **código de recuperación**.
- **Qué significa.** El identificador no lleva datos civiles (nombre, documento): es un UUID aleatorio. El código **solo se muestra esta vez**; el servidor guarda únicamente un resumen (§19.3).

16.2 Lo que ofrece el dispositivo

```
$ OUT=$(WALLET tour-wallet --secrets-out secrets-wallet.txt); echo "$OUT"
# ... (extracto del recorrido: sección 2; el resto del recorrido se muestra en las demás secciones)
— 2. Qué ofrece el custodio del dispositivo (el contrato NO tiene ninguna operación que devuelva una clave privada)
  custodio              : dispositivo-simulado-tee
  nivel máximo          : TEE
  informe               : DeviceReport(custodian=dispositivo-simulado-tee, platform=JVM
(simulado), maxProtection=TEE, hardwareBacked=true)
  attestation           -> AttestationEvidence
  delete                -> void
  descriptor            -> KeyDescriptor
  generate               -> KeyDescriptor
  generate$default      -> KeyDescriptor
  getMaxProtection       -> ProtectionLevel
  getName               -> String
  publicKey             -> ECPublicKey
  report                -> DeviceReport
  seal                  -> byte[]
  sign                  -> byte[]
  unseal                 -> byte[]
```

- **Qué hace.** Muestra el informe del custodio y la lista de operaciones del contrato `KeyCustodian`.
- **Qué significa.** `dispositivo-simulado-tee`, plataforma «JVM (simulado)». Anota la lista de métodos: `generate`, `sign`, `seal`, `unseal`, `publicKey`, `attestation`, `descriptor`, `report`, `delete`. **Ninguno devuelve una clave privada** (se verá en §17).

16.3 La activación

```
$ OUT=$(WALLET tour-wallet --secrets-out secrets-wallet.txt); echo "$OUT"
# ... (extracto del recorrido: sección 3; el resto del recorrido se muestra en las demás secciones)
— 3. Activación de la instancia (desafío de un solo uso + attestation + prueba de posesión)
HTTP 201
{
  "instanceId": "5fa54886-f526-4593-ba89-3666e4309ad5",
  "status": "ACTIVE",
  "accessToken": "GRVBfph1NbrhsGQhTgW7G0xAQud4h7EqEgqXE6YiFqY",
  "protection": {
    "declared": "TEE",
    "verified": "TEE",
    "attested": true
  }
}
```

```

},
"activationRecord": {
  "auditId": 118,
  "activatedAt": "2026-10-01 19:16:28.279817+00",
  "citizenRef": "b49dbd5e-4ae4-4ea5-98cf-6b01db443cce"
}
}

```

- **Qué hace.** Ejecuta la secuencia de §11.3: desafío → generación de clave → prueba de posesión → POST /instances.
- **Qué significa.**
 - 201 y status: ACTIVE: la instancia quedó registrada y activa.
 - protection: declared: TEE, verified: TEE, attested: true (se explica en §18).
 - activationRecord: el **registro de activación** que pide el criterio: el identificador de la entrada de auditoría (auditId), el instante (activatedAt) y el ciudadano al que quedó asociada (citizenRef , el mismo del alta).

16.4 La ficha de la instancia

```

$ OUT=$(WALLET tour-wallet --secrets-out secrets-wallet.txt); echo "$OUT"
# ... (extracto del recorrido: sección 4; el resto del recorrido se muestra en las demás secciones)
— 4. Ficha técnica de la instancia
HTTP 200
{
  "instanceId": "5fa54886-f526-4593-ba89-3666e4309ad5",
  "citizenRef": "b49dbd5e-4ae4-4ea5-98cf-6b01db443cce",
  "status": "ACTIVE",
  "protection": {
    "declared": "TEE",
    "verified": "TEE",
    "attested": true,
    "policyMinimum": "SOFTWARE"
  },
  "thumbprint": "rm4Jnnb6FXa6CKN3A45W7J4HlageC72085zsc0e15TA",
  "publicKey": {
    "x": "yzZ70mfmpPTzPG4DaUs0YTeX1aFZpcB_ypdfrBwulNg",
    "y": "Dvqg4omS8pCleM3OuW1leNtyAM_fIw38nYFWG_qlbcA",
    "crv": "P-256",
    "kty": "EC"
  },
  "deviceProfile": {
    "deviceInfo": {
      "platform": "JVM (simulado)",
      "custodian": "dispositivo-simulado-tee",
      "declaredByDevice": true
    },
    "attestation": {
      "notes": [],
      "origin": 0,
      "ecCurve": 1,
      "present": true,
      "purposes": [
        2
      ],
      "deviceLocked": true,
      "verifiedBootState": 0,
      "attestationVersion": 4,
      "keymasterSecurityLevel": 1,
      "attestationSecurityLevel": 1
    },
    "declaredLevel": "TEE",
    "verifiedLevel": "TEE",

```

```

    "policyMinimumLevel": "SOFTWARE",
    "privateKeyHeldByBackend": false
  },
  "activatedAt": "2026-10-01 19:16:28.279817+00"
}

```

- **Qué significa.** La instancia está **asociada al ciudadano** (`citizenRef`), con su **clave pública** (solo `x`, `y`: no hay componente privado `d`), su huella, y la ficha del dispositivo. La propiedad `privateKeyHeldByBackend: false` es explícita.

16.5 El registro de auditoría

```

$ OUT=$(WALLET tour-wallet --secrets-out secrets-wallet.txt); echo "$OUT"
# ... (extracto del recorrido: sección 5; el resto del recorrido se muestra en las demás secciones)
— 5. Registro de activación (auditoría inalterable)
HTTP 200
[
  {
    "id": 116,
    "at": "2026-10-01 19:16:28.146991+00",
    "actor": "system",
    "action": "CITIZEN_CREATED",
    "detail": {}
  },
  {
    "id": 117,
    "at": "2026-10-01 19:16:28.190278+00",
    "actor": "citizen",
    "action": "CHALLENGE_ISSUED",
    "detail": {
      "purpose": "ACTIVATION"
    }
  },
  {
    "id": 118,
    "at": "2026-10-01 19:16:28.279817+00",
    "actor": "citizen",
    "action": "INSTANCE_ACTIVATED",
    "detail": {
      "attested": true,
      "thumbprint": "rm4Jnnb6FXa6CKN3A45W7J4HlageC72085zsc0e15TA",
      "declaredLevel": "TEE",
      "verifiedLevel": "TEE"
    }
  }
]

```

La auditoría muestra, en orden, `CITIZEN_CREATED` → `CHALLENGE_ISSUED` → `INSTANCE_ACTIVATED`, cada una con su instante.

16.6 Una sola cartera activa por ciudadano

```

$ OUT=$(WALLET tour-wallet --secrets-out secrets-wallet.txt); echo "$OUT"
# ... (extracto del recorrido: sección 6g; el resto del recorrido se muestra en las demás secciones)
— 6g. Segunda cartera activa para el mismo ciudadano
HTTP 409
{
  "error": "ACTIVE_INSTANCE_EXISTS",
  "message": "El ciudadano ya tiene una cartera activa; use la recuperación para reemplazarla"
}

```

- **Qué significa.** 409 ACTIVE_INSTANCE_EXISTS: el ciudadano ya tiene una cartera activa. Para cambiar de dispositivo hay que usar la recuperación (§19).

¿Quién lo impide? **También la base de datos**, no solo la aplicación:

```
$ echo "select indexdef from pg_indexes where indexname='wallet_one_active_per_citizen'" | $PSQL -tA
CREATE UNIQUE INDEX wallet_one_active_per_citizen ON public.wallet_instances USING btree (citizen_id)
WHERE (status = 'ACTIVE'::text)
```

- **Qué hace.** Muestra la definición del índice `wallet_one_active_per_citizen`.
- **Qué significa.** Es un **índice único parcial**: `UNIQUE (citizen_id) WHERE status = 'ACTIVE'`. La base **no permite** dos filas activas para el mismo ciudadano, aunque un error de programación lo intentara.

16.7 Lo que quedó en la base

```
$ echo "select left(id::text,8) as instancia, status, declared_level as declarado, verified_level as
verificado, attested, to_char(activated_at,'HH24:MI:SS') as activada, revoked_reason from
wallet_instances order by activated_at desc limit 4" | $PSQL
instancia | status | declarado | verificado | attested | activada | revoked_reason
-----+-----+-----+-----+-----+-----+-----
ec730742 | ACTIVE | TEE       | TEE       | t       | 19:16:28 |
5fa54886 | REVOKED | TEE       | TEE       | t       | 19:16:28 | RECOVERY
be598a31 | ACTIVE | TEE       | TEE       | t       | 19:15:27 |
d83ca40f | REVOKED | TEE       | TEE       | t       | 19:15:27 | RECOVERY
(4 rows)
```

Cuatro instancias recientes: tres `ACTIVE` (de distintos ciudadanos) y una `REVOKED` con `RECOVERY` (la cartera sustituida en la recuperación, §19). Nótese que `SOFTWARE / attested = f` corresponde al dispositivo de software de otros laboratorios.

16.8 La auditoría no se puede alterar

```
$ echo "select action, count(*) from wallet_audit group by action order by 1" | $PSQL
$ echo "update wallet_audit set action='X' where id = (select min(id) from wallet_audit)" | $PSQL
2>&l | head -2
      action          | count
-----+-----
ACTIVATION_REJECTED   |    18
ATTESTATION_REJECTED  |    12
AUTH_DENIED           |     4
CHALLENGE_ISSUED      |    46
CITIZEN_CREATED       |    32
DID_BACKUP_STORED     |     2
DID_PUBLICATION_STARTED |     2
DID_PUBLISHED         |     2
INSTANCE_ACTIVATED    |     9
INSTANCE_ACTIVATED_BY_RECOVERY |     4
RECOVERY_DENIED       |    10
RECOVERY_STARTED      |     4
(12 rows)

ERROR:  tabla append-only: UPDATE no permitido sobre wallet_audit
CONTEXT:  PL/pgSQL function wallet_forbid_mutation() line 3 at RAISE
```

- **Primera tabla:** recuento de cada tipo de evento (`INSTANCE_ACTIVATED`, `ACTIVATION_REJECTED`, `ATTESTATION_REJECTED`, `RECOVERY_DENIED`...). Se registran también los

rechazos.

- **Segunda orden:** intenta modificar un evento; la base responde `tabla append-only: UPDATE` no permitido sobre `wallet_audit`.

🎯 Conclusión — evidencia del criterio 1

Pregunta	Evidencia
¿Registrada y activa?	201 ACTIVE + ficha
¿Asociada al ciudadano?	citizenRef en el registro de activación y en la ficha
¿Registro de activación?	activationRecord + INSTANCE_ACTIVATED en la auditoría inalterable
¿Una sola activa?	409 + índice único parcial en la base

17. CRITERIO 2 — El par de claves se genera en el hardware seguro y no puede extraerse

Criterio. "El par de claves se genera dentro del hardware seguro y la clave privada no puede extraerse ni exportarse; evidencia: prueba de no exportabilidad."

🔧 Predice antes de ejecutar

(1) ¿Cómo se demuestra que un contrato **no ofrece** una forma de sacar la clave? (2) Si buscamos las claves privadas reales usadas en el laboratorio dentro de **toda** la base de datos, ¿cuántas coincidencias esperas?

17.1 El contrato

```
$ sed -n '46,75p' /home/geovani/Descargas/generic/bitacora/new_proyect/wallet-core/src/main/kotlin/co/org/avance/ssi/wallet/core/KeyCustodian.kt | cut -c1-200
/**
 * HARDWARE SEGURO (abstracción). Contrato de NO EXPORTABILIDAD (ERSo 001, criterio 2):
 * - ningún método devuelve una clave privada ni material equivalente (`PrivateKey`, `KeyPair`, `SecretKey`);
 * - la única operación con la clave privada es `sign`, que recibe datos y devuelve una firma;
 * - la prueba `NonExportabilityTest` verifica esto sobre la firma de la interfaz y sobre el comportamiento.
 * En Android se implementa con Android Keystore (StrongBox/TEE); en iOS con Secure Enclave. Aquí solo hay una simulación en software.
 */
interface KeyCustodian {
    val name: String
    /** Máximo nivel que este custodio realmente puede ofrecer (nunca exagerado). */
    val maxProtection: ProtectionLevel

    /** Genera un par DENTRO del custodio. Si `maxProtection` < `policy.minimumLevel` lanza [ProtectionBelowPolicyException]. */
    fun generate(alias: String, policy: KeyPolicy = KeyPolicy(), attestationChallenge: ByteArray? = null): KeyDescriptor
    fun descriptor(alias: String): KeyDescriptor?
    fun publicKey(alias: String): ECPublicKey

    /** ES256: devuelve r||s (64 bytes). Único uso de la clave privada. */
    fun sign(alias: String, data: ByteArray): ByteArray
```

```

/** Evidencia generada al crear la clave con `attestationChallenge`; null si este custodio no
puede demostrar nada. */
fun attestation(alias: String): AttestationEvidence?

/** Sellado de datos locales con una clave interna del custodio (AES-GCM); la clave de sellado
tampoco sale. */
fun seal(plaintext: ByteArray, aad: ByteArray): ByteArray
fun unseal(sealed: ByteArray, aad: ByteArray): ByteArray

fun delete(alias: String)
fun report(): DeviceReport
}

```

- **Qué hace.** Muestra el código del contrato.
- **Qué significa.** Léelo como una lista de permisos: *generar, consultar el descriptor, obtener la clave **pública**, firmar* (devuelve la firma, no la clave), *atestación, sellar/desellar y borrar*. **No existe** `exportPrivateKey`, `getPrivateKey` ni nada parecido; el único uso de la clave privada es `sign`.
- ⚠ El comentario menciona una prueba `NonExportabilityTest`; en el repositorio esa comprobación vive en `WalletCoreTest` (pruebas `001-C2 ...`, §20).

17.2 Lo que el custodio entrega al crear una clave

El descriptor de una clave (`KeyDescriptor`) incluye `origin = GENERATED` y `exportable = false`. El servidor **rechaza** cualquier clave que no haya nacido dentro:

```

$ OUT=$(WALLET tour-wallet --secrets-out secrets-wallet.txt); echo "$OUT"
# ... (extracto del recorrido: sección 6e; el resto del recorrido se muestra en las demás secciones)
— 6e. Clave IMPORTADA (no generada en el hardware)
HTTP 422
{
  "error": "KEY_NOT_GENERATED_IN_HARDWARE",
  "message": "La clave no fue generada dentro del entorno seguro (origin=2); no se puede declarar no
exportable"
}

```

- **Qué hace.** Un dispositivo cuya attestation dice que la clave fue **importada** (`origin=2`) intenta activarse.
- **Qué significa.** `422 KEY_NOT_GENERATED_IN_HARDWARE`: una clave importada **nunca** puede afirmarse no exportable, porque alguien la tuvo en claro antes de entrar.

17.3 El servidor no guarda ni puede guardar claves privadas

```

$ echo "select table_name, column_name from information_schema.columns where table_name like
'wallet%' and (column_name ~* 'priv|secret|seed|d_value|pkcs8' ) order by 1,2" | $PSQL
$ echo "select table_name, string_agg(column_name, ', ' order by ordinal_position) as columns from
information_schema.columns where table_name='wallet_instances' group by 1" | $PSQL
table_name | column_name
-----+-----
(0 rows)

table_name |
columns
-----+-----
wallet_instances | id, citizen_id, status, public_jwk, thumbprint, declared_level, verified_level,

```

```
attested, device_profile, token_hash, activated_at, revoked_at, revoked_reason, replaced_by
(1 row)
```

- **Primera consulta.** Busca, en **todas** las tablas `wallet_*`, columnas cuyo nombre sugiera material privado (`priv`, `secret`, `seed`, `pkcs8`...). Resultado: **0 filas**.
- **Segunda consulta.** Lista las columnas de `wallet_instances`: hay `public_jwk` (clave pública), `thumbprint` y `token_hash` (resumen del token). Nada privado.

17.4 La prueba definitiva: buscar las claves en toda la base

```
$ docker compose exec -T postgres pg_dump -U $POSTGRES_USER $POSTGRES_DB > $W/dump.sql
$ echo "control: el volcado contiene datos de la cartera -> $(grep -c 'INSTANCE_ACTIVATED'
$W/dump.sql) líneas con INSTANCE_ACTIVATED"
$ echo "formas textuales de claves privadas de prueba a buscar: $(wc -l < $W/secrets-wallet.txt)"
$ echo "coincidencias de ESAS formas en todo el volcado de la base: $(grep -c -F -f $W/secrets-
wallet.txt $W/dump.sql)"
$ rm -f $W/dump.sql
control: el volcado contiene datos de la cartera -> 13 líneas con INSTANCE_ACTIVATED
formas textuales de claves privadas de prueba a buscar: 60
coincidencias de ESAS formas en todo el volcado de la base: 0
```

- **Qué hace.**
 1. El recorrido guardó en un archivo **60 formas textuales** de todas las claves privadas reales que usó (decimal, hexadecimal, base64, base64url, PKCS#8...).
 2. `pg_dump` vuelca **toda la base de datos** a un archivo.
 3. `grep -c -F -f` cuenta cuántas de esas formas aparecen en el volcado.
- **Qué significa.** El **control** (`INSTANCE_ACTIVATED` aparece) demuestra que el volcado contiene datos reales y la búsqueda no está vacía por error. El resultado: **0 coincidencias**. Ninguna clave privada llegó a la base del servidor.
- El archivo del volcado se borra al final.

17.5 Las pruebas automáticas del criterio

Las pruebas `WalletCoreTest > 001-C2 ...` verifican, entre otras cosas: que el contrato no expone **ningún método que devuelva material privado** (por reflexión sobre la interfaz), que la clave sirve para firmar y la firma se verifica con la pública, y que la clave privada real **no aparece en ninguna salida del custodio** (descriptores, firmas). Se listan en §20.

🎯 Conclusión — evidencia del criterio 2

Evidencia	Resultado
Contrato sin métodos que devuelvan clave privada	✓
Clave importada rechazada	✓ KEY_NOT_GENERATED_IN_HARDWARE
Sin columnas privadas en el servidor	✓ 0 filas
60 formas de claves privadas buscadas en toda la base	✓ 0 coincidencias
Prueba en un chip real	⚠ pendiente (§26)

18. CRITERIO 3 — El nivel declarado coincide con el realmente disponible

Criterio. "El nivel de protección declarado coincide con el realmente disponible en el dispositivo; evidencia: ficha técnica y verificación del dispositivo."

Predice antes de ejecutar

(1) Un dispositivo de software declara «TEE». ¿Qué responde el servidor? (2) ¿Y si un dispositivo que sí tiene TEE declara «SOFTWARE» (menos de lo que tiene)? (3) ¿Qué pasa si presenta una attestation firmada por una autoridad que el servidor no conoce?

18.1 La ficha técnica del dispositivo legítimo

```
$ OUT=$(WALLET tour-wallet --secrets-out secrets-wallet.txt); echo "$OUT"
# ... (extracto del recorrido: sección 4; el resto del recorrido se muestra en las demás secciones)
— 4. Ficha técnica de la instancia
HTTP 200
{
  "instanceId": "5fa54886-f526-4593-ba89-3666e4309ad5",
  "citizenRef": "b49dbd5e-4ae4-4ea5-98cf-6b01db443cce",
  "status": "ACTIVE",
  "protection": {
    "declared": "TEE",
    "verified": "TEE",
    "attested": true,
    "policyMinimum": "SOFTWARE"
  },
  "thumbprint": "rm4Jnnb6FXa6CKN3A45W7J4HlageC72085zsc0e15TA",
  "publicKey": {
    "x": "yzZ70mfmpPTzPG4DaUs0YTeX1aFZpcB_ypdfBwulNg",
    "y": "Dvqg4omS8pCleM30uW1leNtyAM_fIw38nYFWG_qlbcA",
    "crv": "P-256",
    "kty": "EC"
  },
  "deviceProfile": {
    "deviceInfo": {
      "platform": "JVM (simulado)",
      "custodian": "dispositivo-simulado-tee",
      "declaredByDevice": true
    },
    "attestation": {
      "notes": [],
      "origin": 0,
      "ecCurve": 1,
      "present": true,
      "purposes": [
        2
      ],
      "deviceLocked": true,
      "verifiedBootState": 0,
      "attestationVersion": 4,
      "keymasterSecurityLevel": 1,
      "attestationSecurityLevel": 1
    },
    "declaredLevel": "TEE",
    "verifiedLevel": "TEE",
    "policyMinimumLevel": "SOFTWARE",
```

```

    "privateKeyHeldByBackend": false
  },
  "activatedAt": "2026-10-01 19:16:28.279817+00"
}

```

En `deviceProfile.attestation` está la **verificación del dispositivo** (lo que el servidor leyó del certificado de la attestation):

Campo	Valor	Significado
<code>present</code>	<code>true</code>	Hubo attestation
<code>attestationSecurityLevel / keymasterSecurityLevel</code>	1	Nivel 1 = TEE (0 = software, 2 = StrongBox)
<code>origin</code>	0	La clave fue generada en el dispositivo
<code>verifiedBootState</code>	0	Arranque verificado
<code>deviceLocked</code>	<code>true</code>	Gestor de arranque bloqueado
<code>purposes</code>	[2]	Propósito: firmar
<code>ecCurve</code>	1	P-256

Y `declaredLevel = TEE`, `verifiedLevel = TEE`: **coinciden**.

18.2 Exagerar el nivel

```

$ OUT=$(WALLET tour-wallet --secrets-out secrets-wallet.txt); echo "$OUT"
# ... (extracto del recorrido: sección 6a; el resto del recorrido se muestra en las demás secciones)
— 6a. Dispositivo de SOFTWARE que declara TEE
HTTP 422
{
  "error": "PROTECTION_LEVEL_OVERSTATED",
  "message": "El nivel declarado (TEE) supera el que el dispositivo demuestra (SOFTWARE)",
  "details": [
    "declared=TEE",
    "verified=SOFTWARE"
  ]
}

```

- **Qué significa.** 422 `PROTECTION_LEVEL_OVERSTATED`: se declaró TEE, pero lo demostrado es `SOFTWARE`. Sin attestation, el servidor **asume lo peor**.

18.3 Declarar de menos

```

$ OUT=$(WALLET tour-wallet --secrets-out secrets-wallet.txt); echo "$OUT"
# ... (extracto del recorrido: sección 6b; el resto del recorrido se muestra en las demás secciones)
— 6b. Dispositivo TEE que declara SOFTWARE (declarar de menos también se rechaza)
HTTP 422
{
  "error": "PROTECTION_LEVEL_UNDERSTATED",
  "message": "El nivel declarado (SOFTWARE) no coincide con el disponible (TEE)",
  "details": [
    "declared=SOFTWARE",
    "verified=TEE"
  ]
}

```

- **Qué significa.** 422 `PROTECTION_LEVEL_UNDERSTATED`. También se rechaza: el criterio dice «**coincide**», no «no supera». (Es la lectura literal; ver §26 si se prefiere aceptar la

declaración a la baja.)

18.4 Una autoridad desconocida

```
$ OUT=$(WALLET tour-wallet --secrets-out secrets-wallet.txt); echo "$OUT"
# ... (extracto del recorrido: sección 6c; el resto del recorrido se muestra en las demás secciones)
— 6c. Attestation firmada por una raíz desconocida
HTTP 422
{
  "error": "ATTESTATION_INVALID",
  "message": "La attestation no es válida",
  "details": [
    "UNTRUSTED_CHAIN",
    "La cadena no llega a una raíz de confianza: Path does not chain with any of the trust anchors"
  ]
}
```

- **Qué significa.** `ATTESTATION_INVALID` / `UNTRUSTED_CHAIN`: la cadena de certificados no llega a la raíz de confianza del servidor.

18.5 Una attestation vieja

```
$ OUT=$(WALLET tour-wallet --secrets-out secrets-wallet.txt); echo "$OUT"
# ... (extracto del recorrido: sección 6d; el resto del recorrido se muestra en las demás secciones)
— 6d. Attestation vieja (otro desafío)
HTTP 422
{
  "error": "ATTESTATION_INVALID",
  "message": "La attestation no es válida",
  "details": [
    "CHALLENGE_MISMATCH",
    "El desafío de la attestation no es el emitido por el backend"
  ]
}
```

- **Qué significa.** `CHALLENGE_MISMATCH`: la attestation lleva incrustado un desafío distinto al que el servidor acaba de emitir. Una prueba capturada antes **no se puede reutilizar**.

18.6 Arranque no verificado

```
$ OUT=$(WALLET tour-wallet --secrets-out secrets-wallet.txt); echo "$OUT"
# ... (extracto del recorrido: sección 6f; el resto del recorrido se muestra en las demás secciones)
— 6f. Arranque no verificado (gestor de arranque desbloqueado)
HTTP 422
{
  "error": "PROTECTION_LEVEL_OVERSTATED",
  "message": "El nivel declarado (TEE) supera el que el dispositivo demuestra (SOFTWARE)",
  "details": [
    "declared=TEE",
    "verified=SOFTWARE"
  ]
}
```

- **Qué significa.** Aunque la attestation sea auténtica, si el gestor de arranque está desbloqueado el servidor **rebaja** el nivel a `SOFTWARE` y, al haber declarado `TEE`, rechaza. Una política del proyecto (`WALLET_REQUIRE_VERIFIED_BOOT`).

Conclusión — evidencia del criterio 3

Intento	Respuesta
Declarar lo real (TEE con attestation)	201, declared = verified = TEE
Software que declara TEE	PROTECTION_LEVEL_OVERSTATED
TEE que declara software	PROTECTION_LEVEL_UNDERSTATED
Raíz desconocida	ATTESTATION_INVALID · UNTRUSTED_CHAIN
Attestation vieja	ATTESTATION_INVALID · CHALLENGE_MISMATCH
Clave importada	KEY_NOT_GENERATED_IN_HARDWARE
Arranque no verificado	nivel rebajado → OVERSTATED

19. CRITERIO 4 — La recuperación de cuenta se completa sin acceder a la clave privada

Criterio. "La recuperación de cuenta se completa desde el backend sin acceder a la clave privada; evidencia: prueba de recuperación."

Predice antes de ejecutar

(1) El dispositivo se pierde. ¿El servidor puede devolver la clave privada? (2) Con el código correcto, ¿qué recibe el ciudadano? (3) ¿Qué pasa con la cartera vieja? (4) ¿Se puede adivinar el código por fuerza bruta?

19.1 Código incorrecto

```
$ OUT=$(WALLET tour-wallet --secrets-out secrets-wallet.txt); echo "$OUT"
# ... (extracto del recorrido: sección 7a; el resto del recorrido se muestra en las demás secciones)
— 7a. Código incorrecto
HTTP 401
{
  "error": "INVALID_RECOVERY_CODE",
  "message": "Código de recuperación inválido"
}
```

401 INVALID_RECOVERY_CODE.

19.2 Código correcto: un token de un solo uso

```
$ OUT=$(WALLET tour-wallet --secrets-out secrets-wallet.txt); echo "$OUT"
# ... (extracto del recorrido: sección 7b; el resto del recorrido se muestra en las demás secciones)
— 7b. Código correcto: token de recuperación de un solo uso
HTTP 200
{
  "recoveryToken": "IRrp9JR5fYofyDHoCHU9aGwbttzFsgciwwMI2BQsdIM",
  "expiresAt": "2026-10-01 19:26:28.761135+00",
  "next": "Active una instancia nueva enviando este recoveryToken"
}
```

- **Qué significa.** El servidor entrega un `recoveryToken` que **vence a los 10 minutos** y solo sirve una vez. No entrega ninguna clave: no la tiene.

19.3 Cómo guarda el servidor el código

```
$ echo "select left(id::text,8) as ciudadano, length(recovery_salt) as sal_bytes,
length(recovery_hash) as hash_bytes, left(encode(recovery_hash,'hex'),16)||'...' as hash,
failed_attempts, locked_until is not null as bloqueado from wallet_citizens order by created_at desc
limit 3" | $PSQL
ciudadano | sal_bytes | hash_bytes | hash | failed_attempts | bloqueado
-----+-----+-----+-----+-----+-----
15a077c9 | 16 | 32 | c5fa82436b11f8be... | 0 | f
78e29d48 | 16 | 32 | fe2275b781257fd6... | 0 | f
47fb8a98 | 16 | 32 | d3f36f524b19577c... | 0 | f
(3 rows)
```

- **Qué significa.** Por cada ciudadano hay una **sal** de 16 bytes y un **resumen** de 32 bytes (PBKDF2). El código no está en la base; con el resumen no se puede reconstruir.

19.4 Activar el dispositivo nuevo

```
$ OUT=$(WALLET tour-wallet --secrets-out secrets-wallet.txt); echo "$OUT"
# ... (extracto del recorrido: sección 7c; el resto del recorrido se muestra en las demás secciones)
— 7c. Dispositivo nuevo se activa con el token
HTTP 201
{
  "instanceId": "ec730742-c6f6-4dc1-95bc-03eec473df4f",
  "status": "ACTIVE",
  "accessToken": "VwbMb2UwUf-WXXMVLU1CQplw_4_m46Wq6qW4Zr4al0s",
  "protection": {
    "declared": "TEE",
    "verified": "TEE",
    "attested": true
  },
  "activationRecord": {
    "auditId": 142,
    "activatedAt": "2026-10-01 19:16:28.819134+00",
    "citizenRef": "b49dbd5e-4ae4-4ea5-98cf-6b01db443cce"
  },
  "recovery": {
    "revokedInstances": [
      "5fa54886-f526-4593-ba89-3666e4309ad5"
    ],
    "reissueRequired": true,
    "note": "Las credenciales estaban ligadas a la clave del dispositivo anterior y deben reemitirse;
el DID anterior no se puede desactivar sin su clave"
  }
}
```

- **Qué significa.**
 - `201 ACTIVE`: el dispositivo nuevo, **con clave nueva**, quedó activo para el mismo ciudadano.
 - `recovery.revokedInstances`: la cartera anterior fue **revocada**.
 - `reissueRequired: true` y la nota: las credenciales estaban ligadas a la clave anterior y **deben reemitirse**. Es la prueba de que **no se recuperó la clave**: se reemplazó.

19.5 La cartera anterior

```
$ OUT=$(WALLET tour-wallet --secrets-out secrets-wallet.txt); echo "$OUT"
# ... (extracto del recorrido: sección 7d; el resto del recorrido se muestra en las demás secciones)
— 7d. La cartera anterior
HTTP 403
{
  "error": "INSTANCE_REVOKED",
  "message": "La instancia fue revocada (RECOVERY)"
}
```

403 INSTANCE_REVOKED: la cartera perdida ya no autentica, aunque alguien la encuentre.

19.6 Reutilizar el token

```
$ OUT=$(WALLET tour-wallet --secrets-out secrets-wallet.txt); echo "$OUT"
# ... (extracto del recorrido: sección 7e; el resto del recorrido se muestra en las demás secciones)
— 7e. Reutilizar el token
HTTP 403
{
  "error": "RECOVERY_TOKEN_INVALID",
  "message": "Token de recuperación inexistente, expirado, usado o de otro ciudadano"
}
• 10 claves privadas de prueba registradas en secrets-wallet.txt (solo laboratorio) para buscarlas
en la base de datos
RESULTADO tour-wallet: 0 PASS • 0 FAIL
```

403 RECOVERY_TOKEN_INVALID: un solo uso.

19.7 Fuerza bruta sobre el código

```
$ REF=$(curl -s $CW -X POST $BASE/wallet/v1/citizens -H "Content-Type: application/json" -d '{}' | jq
-r .citizenRef); echo "ciudadano de prueba: $REF"
$ for i in 1 2 3 4 5 6; do printf "intento %s con un código falso -> " $i; curl -s $CW -X POST
$BASE/wallet/v1/recovery/start -H "Content-Type: application/json" -d "
{"citizenRef\":\"$REF\", \"recoveryCode\":\"AAAA-BBBB-CCCC-DDDD-EEE$i\"}" | jq -c
'[,error,message]'; done
$ echo "select failed_attempts, locked_until is not null as bloqueado from wallet_citizens where
id='$REF'" | $PSQL
ciudadano de prueba: 307f2d62-fcee-4c8c-bb56-4c21d5a7144e
intento 1 con un código falso -> ["INVALID_RECOVERY_CODE","Código de recuperación inválido"]
intento 2 con un código falso -> ["INVALID_RECOVERY_CODE","Código de recuperación inválido"]
intento 3 con un código falso -> ["INVALID_RECOVERY_CODE","Código de recuperación inválido"]
intento 4 con un código falso -> ["INVALID_RECOVERY_CODE","Código de recuperación inválido"]
intento 5 con un código falso -> ["INVALID_RECOVERY_CODE","Código de recuperación inválido"]
intento 6 con un código falso -> ["RECOVERY_LOCKED","Demasiados intentos; intente más tarde"]
failed_attempts | bloqueado
-----+-----
5 | t
(1 row)
```

- **Qué hace.** Crea un ciudadano de prueba y envía 6 códigos falsos.
- **Qué significa.** Los 5 primeros: INVALID_RECOVERY_CODE. El sexto: RECOVERY_LOCKED («Demasiados intentos»). La base muestra failed_attempts = 5 y bloqueado = t: bloqueo de 15 minutos. Adivinar un código de 20 caracteres a 5 intentos por cuarto de hora es inviable.

19.8 Ninguna clave privada tocó al servidor

La búsqueda de §17.4 incluye las claves privadas de **ambos** dispositivos (el perdido y el nuevo): 0 coincidencias.

Conclusión — evidencia del criterio 4

Paso	Evidencia
Código erróneo	401
Código correcto	token de un solo uso (10 min)
Dispositivo nuevo	201 ACTIVE , clave nueva
Cartera anterior	REVOKED
Credenciales	reissueRequired: true
Token reutilizado	403
Fuerza bruta	bloqueo tras 5 intentos
Claves privadas en el servidor	0

20. Pruebas automáticas

```
$ cd /home/geovani/Descargas/generic/bitacora/new_proyect
$ TEST_DB_URL=jdbc:postgresql://127.0.0.1:55432/wallet_test ./gradlew :wallet-core:test :wallet-
service:test --rerun-tasks -q 2>&1 | tail -3; echo '(sin salida = todas pasaron)'
(sin salida = todas pasaron)
$ python3 - <<'EOF'
> import re, glob, html
> for pat in ['wallet-core/build/test-results/test/*WalletCoreTest*.xml', 'wallet-service/build/test-
results/test/*KeyAttestationTest*.xml', 'wallet-service/build/test-
results/test/*Erso001WalletTest*.xml']:
>     for f in sorted(glob.glob(pat)):
>         s = open(f, encoding='utf-8').read()
>         tot = re.search(r'name="([^\"]+)" tests="(\d+)" skipped="(\d+)" failures="(\d+)"',
s).groups()
>         print(f"{tot[0].split('.')[1]}: {tot[1]} pruebas, {tot[2]} omitidas, {tot[3]} fallos")
>         for m in re.finditer(r'<testcase name="([^\"]+)"[^>]*?(/>|>(.*)</testcase>)', s, re.S):
>             name = html.unescape(m.group(1)).removesuffix("(")
>             estado = "FALLA" if "<failure" in (m.group(3) or "") else "PASA "
>             print(f"    [{estado}] {name}")
>         print()
> EOF
WalletCoreTest: 15 pruebas, 0 omitidas, 0 fallos
[PASA ] 001-C3 la politica minima permite el nivel real y lo refleja
[PASA ] 001-C2 el contrato del custodio no expone ningun metodo que devuelva material privado
[PASA ] 001-C2 la clave sirve para firmar y la firma se verifica con la publica, sin salir del
custodio
[PASA ] 003-C2 el DID Document es conforme y produce un informe de conformidad
[PASA ] 001-C2 la clave privada real no aparece en ninguna salida del custodio, ni en descriptores,
ni en firmas
[PASA ] 003 la validacion es un requisito previo - un documento no conforme no se entrega
[PASA ] 003 el SDK firma con la clave del DID usando el custodio
[PASA ] 003-C3 authentication y assertionMethod apuntan a la clave Multikey del titular
[PASA ] guarda - todo metodo de prueba devuelve void para que JUnit lo ejecute
[PASA ] 001-C3 un custodio de software declara SOFTWARE y no puede pasar por hardware
[PASA ] 001-C2 no se puede sobrescribir una clave existente ni obtener una inexistente
[PASA ] 003-C4 el documento se almacena sellado y asociado a la instancia de cartera
[PASA ] 003-C1 el DID se genera en el dispositivo sin exponer la clave privada (registro de
```

```

generacion)
[PASA ] 003-C2 el informe detecta documentos no conformes
[PASA ] 003 el identificador se deriva de la clave publica y es determinista

KeyAttestationTest: 7 pruebas, 0 omitidas, 0 fallos
[PASA ] una cadena firmada por una raiz desconocida se rechaza
[PASA ] una cadena TEE legitima se verifica y entrega el nivel, el origen y la clave
[PASA ] guarda - todo metodo de prueba devuelve void para que JUnit lo ejecute
[PASA ] StrongBox y Software se distinguen
[PASA ] una clave importada queda visible en el origen
[PASA ] un desafio distinto se rechaza (evita reutilizar una attestation vieja)
[PASA ] entradas invalidas se rechazan sin excepciones inesperadas

Erso001WalletTest: 15 pruebas, 0 omitidas, 0 fallos
[PASA ] C1 el titular puede revocar su instancia y deja de autenticar
[PASA ] C3 la politica minima del backend rechaza carteras por debajo del nivel exigido
[PASA ] C1 una segunda cartera activa para el mismo ciudadano se rechaza, y la base lo garantiza
[PASA ] C3 attestation ilegítima - clave importada, raiz desconocida y desafio reutilizado
[PASA ] C1 la instancia queda registrada y activa asociada al ciudadano, con registro de activacion
[PASA ] C4 la recuperacion se completa desde el backend sin acceder a ninguna clave privada
[PASA ] la auditoria del backend es inalterable
[PASA ] el inventario de endpoints del backend solo contiene gestion de cartera y de DID, sin
protocolos de credenciales
[PASA ] C4 el codigo de recuperacion se protege contra fuerza bruta
[PASA ] C3 un dispositivo que exagera su nivel es rechazado - sin evidencia, con evidencia de
software o con arranque no verificado
[PASA ] guarda - todo metodo de prueba devuelve void para que JUnit lo ejecute
[PASA ] C4 el token de recuperacion es de un solo uso y solo sirve para su ciudadano
[PASA ] C3 declarar menos de lo que se tiene tampoco coincide
[PASA ] C1 desafio de un solo uso, prueba de posesion obligatoria y autenticacion de instancia
[PASA ] C3 un dispositivo TEE con attestation valida se activa con el nivel verificado y ficha
tecnica

```

- **Qué hace.** Ejecuta las pruebas de `wallet-core` y `wallet-service` (el servicio contra una base desechable, `vdr-test-pg`, puerto 55432).
- **Qué significa.** Se muestran las tres clases de esta ERSO: `WalletCoreTest` (el contrato y el custodio; incluye también pruebas de la ERSO 003), `KeyAttestationTest` (la verificación de attestation) y `Erso001WalletTest` (el servicio de extremo a extremo). Sin fallos.

Prueba	Criterio
WalletCoreTest > 001-C2 ... (4 pruebas)	2
001-C3 ... y KeyAttestationTest	3
Erso001WalletTest > C1 ...	1
Erso001WalletTest > C3 ...	3
Erso001WalletTest > C4 ...	4

21. Recorrido de extremo a extremo (extracto del registro)

El script `scripts/e2e.sh` ejecuta todo el proyecto (121 comprobaciones). Este es el bloque de la ERSO 001:

```

ERSo 2026-001 · Cartera de identidad y custodia de claves en hardware (dispositivo SIMULADO +
Wallet Backend real)

· Dispositivo TEE simulado, attestation firmada por la autoridad de laboratorio; el backend real

```

verifica la cadena

- ✓ PASS C1 la instancia queda registrada y activa (201, ACTIVE)
- ✓ PASS C3 nivel declarado = nivel verificado por attestation = TEE
- ✓ PASS C1 la instancia queda asociada al ciudadano
- ✓ PASS C3 ficha técnica: la clave nació dentro del entorno seguro (origin=GENERATED) con arranque

verificado

- ✓ PASS C1 registro de activación en la auditoría
- ✓ PASS C3 un dispositivo de software que declara TEE es rechazado (PROTECTION_LEVEL_OVERSTATED)
- ✓ PASS C3 attestation firmada por una raíz desconocida es rechazada (UNTRUSTED_CHAIN)
- ✓ PASS C3 reutilizar una attestation con otro desafío es rechazado (CHALLENGE_MISMATCH)
- ✓ PASS C2 una clave IMPORTADA (no generada en hardware) no puede declararse no exportable
- ✓ PASS un ciudadano no puede tener dos carteras activas (409)
 - Recuperación de cuenta: el dispositivo se perdió
- ✓ PASS C4 código de recuperación incorrecto → 401
- ✓ PASS C4 código correcto → token de recuperación de un solo uso
- ✓ PASS C4 el dispositivo nuevo se activa con el token (201)
- ✓ PASS C4 la cartera anterior quedó revocada
- ✓ PASS C4 se informa que las credenciales deben reemitirse (ligadas a la clave anterior)
- ✓ PASS C4 el token de recuperación no se puede reutilizar
 - 8 claves privadas de prueba registradas en secrets-wallet.txt (solo laboratorio) para buscarlas

en la base de datos

RESULTADO wallet: 16 PASS · 0 FAIL

- ✓ PASS control: el volcado de la base contiene datos de la cartera (la búsqueda no es vacía)
- ✓ PASS C2/C4 ninguna clave privada de los dispositivos aparece en la base de datos del backend

(pg_dump)

- ✓ PASS la auditoría de la cartera es inalterable (trigger append-only)
- ✓ PASS una sola cartera activa por ciudadano: lo garantiza la base (índice único parcial)

PARTE V — PREGUNTAS Y RESPUESTAS

22. Preguntas sobre los conceptos

- 1. ¿Qué significa que una clave sea «no exportable»?** Que puede **usarse** (para firmar) pero no **leerse** ni copiarse. El hardware seguro recibe datos, firma por dentro y devuelve solo la firma.
- 2. ¿Por qué una clave importada no puede declararse no exportable?** Porque, antes de entrar al hardware, existió fuera de él: alguien (la aplicación, el sistema) la tuvo en claro y pudo copiarla. Solo una clave **generada dentro** puede dar esa garantía.
- 3. ¿Qué diferencia hay entre TEE y StrongBox?** El TEE es una zona aislada del procesador principal; StrongBox es un **chip de seguridad independiente**, con protección física adicional. Android reporta ambos en la key attestation.
- 4. ¿Por qué «pedir StrongBox» no basta?** Es una petición: si el aparato no lo tiene, el sistema puede caer a TEE o a software **sin avisar**. Por eso el servidor no confía en la petición sino en la **attestation**, que es lo que el hardware mismo certifica.
- 5. ¿Qué es la key attestation y por qué se puede confiar en ella?** Un certificado producido por el hardware al crear la clave, con una cadena que termina en una **raíz del fabricante/Google**. Se puede confiar porque el servidor tiene la raíz de antemano y verifica la cadena; un dispositivo falso no puede firmar con la clave del fabricante.
- 6. ¿Para qué sirve el desafío dentro de la attestation?** Para impedir la **repetición**: el servidor entrega un valor de un solo uso, el dispositivo lo incrusta al crear la clave, y el servidor comprueba que coincide. Una attestation de otro día no sirve.
- 7. ¿Por qué se exige también una prueba de posesión?** La attestation prueba que **existe** una clave de ese nivel; la prueba de posesión (firmar el desafío) prueba que quien activa **controla** esa clave concreta.
- 8. ¿Por qué el servidor rechaza declarar de menos?** El criterio dice que el nivel declarado **coincide** con el real. Aceptar una declaración a la baja sería más permisivo; el proyecto eligió la lectura literal. Si el grupo prefiere aceptarla, es un cambio de una línea (§26).
- 9. ¿Por qué el arranque no verificado rebaja el nivel?** Si el sistema operativo pudo ser alterado, no se puede confiar en que las operaciones «protegidas» lo sean. Es una decisión de política (`WALLET_REQUIRE_VERIFIED_BOOT`).
- 10. ¿Por qué solo una cartera activa por ciudadano y por qué lo garantiza la base de datos?** Evita ambigüedad sobre cuál es «la» cartera. Se pone en la base (índice único parcial) porque una regla solo en la aplicación puede fallar por un error o una condición de carrera; la base la hace cumplir siempre.
- 11. ¿Por qué la recuperación no devuelve la clave?** Porque el servidor nunca la tuvo. Recuperar es **reemplazar**: clave nueva, cartera nueva, la vieja revocada.

12. ¿Por qué hay que reemitir las credenciales tras recuperar? Las credenciales se ligan a la clave del titular (`cnf`). Con una clave nueva, las antiguas ya no se pueden presentar.

13. ¿Qué pasa con el DID publicado con la clave perdida? Queda **huérfano**: para desactivarlo o rotarlo haría falta firmar con esa clave (ERSo 008). Es un problema abierto (§26).

14. ¿Por qué el código de recuperación se guarda como resumen con sal y PBKDF2? Para que, si la base se filtra, no se pueda obtener el código; PBKDF2 es lento a propósito y la sal impide tablas precalculadas.

15. ¿Qué evita el bloqueo por intentos? La fuerza bruta sobre el código: 5 intentos y 15 minutos de bloqueo.

23. Preguntas por laboratorio y por criterio

Criterio 1 (laboratorio A)

P1. *¿Qué contiene el «registro de activación»?* → `auditId`, `activatedAt` y `citizenRef`; y una fila `INSTANCE_ACTIVATED` en la auditoría. **P2.** *Segunda cartera activa:* → `409 ACTIVE_INSTANCE_EXISTS`; además la impide la base con el índice único parcial. **P3.** *¿Se puede editar la auditoría?* → No: `tabla append-only: UPDATE no permitido`. **P4.** *¿El identificador del ciudadano lleva datos civiles?* → No, es un UUID aleatorio.

Criterio 2 (laboratorio B)

P5. *¿Cómo se demuestra que el contrato no ofrece forma de extraer la clave?* → Leyendo la interfaz (ninguna operación devuelve una clave privada), y con la prueba por reflexión `001-C2` que lo comprueba automáticamente. **P6.** *Búsqueda de claves privadas en la base: ¿qué resultado y qué valor tiene el «control»?* → 0 coincidencias de 60 formas buscadas; el control (`INSTANCE_ACTIVATED` aparece 13 veces en el volcado) prueba que la búsqueda no era vacía. **P7.** *¿Por qué se buscan 60 formas y no una?* → Una clave puede aparecer escrita de muchas maneras (decimal, hex, base64, PKCS#8...); se buscan todas. **P8.** *¿Qué no demuestra este laboratorio?* → Que un **chip real** se niegue a exportar la clave; requiere un dispositivo (§26).

Criterio 3 (laboratorio C)

P9. *Tabla de respuestas.*

Situación	Respuesta
Declaración correcta	<code>201, declared = verified</code>
Software declarando TEE	<code>PROTECTION_LEVEL_OVERSTATED</code>
TEE declarando software	<code>PROTECTION_LEVEL_UNDERSTATED</code>
Raíz desconocida	<code>ATTESTATION_INVALID · UNTRUSTED_CHAIN</code>
Attestation vieja	<code>ATTESTATION_INVALID · CHALLENGE_MISMATCH</code>
Clave importada	<code>KEY_NOT_GENERATED_IN_HARDWARE</code>
Arranque no verificado	nivel rebajado → <code>OVERSTATED</code>

P10. *¿Qué campos de la ficha prueban la «verificación del dispositivo»?* → `attestationSecurityLevel`, `origin`, `verifiedBootState`, `deviceLocked`, `purposes` y los niveles `declared/verified`. **P11.** *¿Por qué `deviceInfo` se rotula `declaredByDevice: true`?* → Porque lo afirma el propio dispositivo; no tiene valor probatorio.

Criterio 4 (laboratorio D)

P12. *¿Qué recibe el ciudadano con el código correcto?* → Un token de un solo uso con vigencia de 10 minutos. **P13.** *¿Qué ocurre con la cartera anterior?* → Se revoca (`REVOKED`, motivo `RECOVERY`); `403 INSTANCE_REVOKED`. **P14.** *¿Qué demuestra `reissueRequired: true`?* → Que la clave no se recuperó, se reemplazó. **P15.** *¿Cuántos intentos fallidos se permiten?* → 5; al sexto, `RECOVERY_LOCKED` durante 15 minutos. **P16.** *¿Cómo se sabe que el servidor no accedió a ninguna clave privada durante la recuperación?* → La búsqueda en la base incluye las claves de ambos dispositivos y da 0.

Pruebas automáticas

P17. *¿Cuántas pruebas hay?* → 15 (`WalletCoreTest`, compartidas con la ERSO 003) + 7 (`KeyAttestationTest`) + 15 (`Erso001WalletTest`).

PARTE VI — CÓMO RESPONDER A CADA CRITERIO

24. Redacción modelo de las cuatro respuestas

Criterio 1 — Instancia registrada y activa, asociada al ciudadano

La activación (`POST /wallet/v1/instances`) exige un desafío de un solo uso, una attestation verificable y una prueba de posesión. El resultado es una instancia `ACTIVE` asociada al ciudadano (`citizenRef`), con su clave pública, su huella y su ficha técnica, y un registro de activación (`activationRecord`, evento `INSTANCE_ACTIVATED` en una auditoría de solo-agregar que la base impide modificar). Una segunda cartera activa del mismo ciudadano se rechaza (`409`) y también lo impide un índice único parcial. **Evidencia:** §16; pruebas

`Erso001WalletTest > C1 ...`.

Criterio 2 — Clave generada en hardware seguro y no exportable

El contrato `KeyCustodian` no ofrece ninguna operación que devuelva una clave privada: solo genera, firma y sella. La clave se genera dentro del custodio (`origin = GENERATED`, `exportable = false`) y el servidor rechaza claves importadas (`KEY_NOT_GENERATED_IN_HARDWARE`). El servidor no tiene columnas de clave privada y una búsqueda de 60 formas textuales de las claves privadas reales en el volcado completo de la base dio 0 coincidencias. **Límite:** el custodio es una simulación; la prueba sobre un chip real queda pendiente. **Evidencia:** §17; pruebas `WalletCoreTest > 001-C2 ...`.

Criterio 3 — Nivel declarado = nivel realmente disponible

El `Wallet Backend` deduce el nivel de la key attestation (cadena hasta una raíz de confianza, desafío de un solo uso, origen y arranque verificado) y lo compara con el declarado: coincidir da `201`; exagerar (`OVERSTATED`), declarar de menos (`UNDERSTATED`), raíz desconocida, attestation vieja, clave importada o arranque no verificado se rechazan. La ficha técnica de la instancia recoge la verificación (`attestationSecurityLevel`, `origin`, `verifiedBootState` ...).

Límite: la attestation se verifica contra una autoridad de laboratorio, no contra las raíces reales de Google. **Evidencia:** §18; pruebas `KeyAttestationTest` y `Erso001WalletTest > C3 ...`.

Criterio 4 — Recuperación sin acceder a la clave privada

Con el código de recuperación (guardado solo como resumen PBKDF2 con sal) el backend entrega un token de un solo uso (10 min); con él se activa un dispositivo nuevo con una clave nueva y la cartera anterior queda revocada; se informa que las credenciales deben reemitirse. El token no se reutiliza y la fuerza bruta se bloquea tras 5 intentos. Ninguna de

las claves privadas (de ambos dispositivos) aparece en la base. **Evidencia:** §19; pruebas Erso001WalletTest > C4 ...

25. Lista de comprobación para quien acepta

☐ Qué comprobar	Cómo	Resultado esperado
☐ Alta del ciudadano	tour-wallet §1	citizenRef + código de recuperación
☐ Activación	tour-wallet §3	201 ACTIVE , activationRecord
☐ Ficha técnica	§4	declared = verified , privateKeyHeldByBackend:false
☐ Auditoría	§5 y wallet_audit	INSTANCE_ACTIVATED presente
☐ Una sola activa	§6g + índice	409 + wallet_one_active_per_citizen
☐ Auditoría inalterable	UPDATE wallet_audit	append-only
☐ Contrato sin exportación	Leer KeyCustodian	Sin método que devuelva clave privada
☐ Clave importada	§6e	KEY_NOT_GENERATED_IN_HARDWARE
☐ Sin columnas privadas	consulta de columnas	0 filas
☐ Claves privadas en la base	pg_dump + grep -F -f	0 coincidencias (control > 0)
☐ Exageración	§6a	OVERSTATED
☐ Declarar de menos	§6b	UNDERSTATED
☐ Raíz falsa	§6c	UNTRUSTED_CHAIN
☐ Attestation vieja	§6d	CHALLENGE_MISMATCH
☐ Arranque no verificado	§6f	nivel rebajado
☐ Código erróneo	§7a	401
☐ Recuperación	§7b-7c	token + 201 , anterior revocada
☐ Token reutilizado	§7e	403
☐ Fuerza bruta	6 intentos	RECOVERY_LOCKED
☐ Pruebas automáticas	./gradlew :wallet-core:test :wallet-service:test	37/37 de la ERSO

PARTE VII — LÍMITES, HALLAZGOS Y OPERACIÓN

26. Lo que este informe NO demuestra, y lo que se encontró

26.1 Límites

Límite	Detalle
No hay hardware seguro real	El custodio es una simulación en software que declara lo que es. Se prueba el contrato y el comportamiento del backend . Falta la prueba en un dispositivo : <code>KeyInfo.isInsideSecureHardware</code> , <code>PrivateKey.encoded == null</code> , intento de exportar. Está descrita como plantilla instrumentada en <code>docs/ANDROID-REFERENCIA.md</code> .
Attestation de laboratorio	Se verificó contra el esquema público de Android pero con cadenas generadas por una autoridad simulada (<code>LabAttestationAuthority</code>). No se probó con cadenas reales de Google ni de fabricantes. Antes de producción: usar como anclas las raíces de Google, validar con dispositivos reales y consultar la lista de revocación.
Igualdad estricta	Declarar de menos también se rechaza (<code>UNDERSTATED</code>). Lectura literal de «coincide»; aceptar la declaración a la baja sería un cambio de una línea.
Nivel mínimo	En laboratorio es <code>SOFTWARE</code> para poder probar. En producción debería ser <code>TEE</code> o superior (<code>WALLET_MIN_LEVEL</code>).
Autenticación del ciudadano	Fuera de esta ERSO. <code>POST /citizens</code> está abierto en laboratorio; en producción el alta debe colgar de un proceso de verificación de identidad.
La recuperación no recupera la clave	Las credenciales deben reemitirse y el DID publicado con la clave perdida no puede desactivarse (haría falta esa clave, como advierte la ERSO 008): queda huérfano. Rotar un DID con la clave vigente perdida es un problema abierto.
Una sola cartera activa	Varios dispositivos simultáneos no se modelan.
Sin limitación de tasa general	Solo la recuperación tiene bloqueo (5 intentos, 15 min).
Autenticación por operación	<code>userAuthenticationRequired</code> (biometría/PIN por firma) no se modela.
deviceInfo	Lo declara el propio dispositivo; sin valor probatorio.
Discrepancia entre fuentes	El informe de Keystore del equipo cita <code>eudi-lib-android-wallet-core 0.28.1</code> ; el repositorio <code>eudi-app-android-wallet-ui</code> declara <code>0.30.2</code> . Son proyectos distintos; no se unificó nada.
Tokens y códigos en las salidas	Los tokens y códigos que aparecen en este informe son de laboratorio y de corta vida.

26.2 Brechas del wallet de referencia EUDI

Resumen del análisis de `eudi-app-android-wallet-ui` (`docs/ANALISIS-BASES-EUDI.md`): la aplicación delega las claves a `eudi-lib-android-wallet-core` y solo **pide** StrongBox; no lee `KeyInfo / securityLevel`, no verifica la attestation y no tiene registro de instancia, recuperación ni respaldo propios. El Wallet Backend de este proyecto cubre esas brechas **del lado del servidor**.

26.3 Hallazgo heredado: HEAD responde 404 en el registro

Descubierto en la ERSO 005. No afecta a esta ERSO.

27. Si algo no sale como en el informe

Síntoma	Causa probable	Qué hacer
<code>holder</code> imprime el texto de uso y no el recorrido	La imagen <code>holder-sim</code> es anterior al comando <code>tour-*</code>	<code>./gradlew :wallet-sim:installDist</code> y <code>docker compose --profile tools build holder</code>
Falta <code>--authority / authority.json</code> no existe	No se creó la autoridad de laboratorio	<code>holder authority-init --out authority.json --root-out lab-attestation-root.pem</code> (el <code>e2e.sh</code> lo hace)
<code>UNTRUSTED_CHAIN</code> en un dispositivo legítimo	El backend no tiene la raíz de la autoridad vigente	Reiniciar <code>wallet</code> tras regenerar <code>lab-attestation-root.pem</code>
401 al activar	Desafío usado o vencido	Repetir desde el alta
<code>ACTIVE_INSTANCE_EXISTS</code> inesperado	El ciudadano ya tiene cartera activa	Usar un ciudadano nuevo o la recuperación
<code>RECOVERY_LOCKED</code>	Demasiados intentos	Esperar 15 minutos o usar otro ciudadano
<code>pg_dump: command not found</code>	Se ejecutó fuera del contenedor	Usar <code>docker compose exec -T postgres pg_dump ...</code>
Pruebas del servicio no conectan	Falta la base desechable	Levantar <code>vdr-test-pg</code> (puerto 55432)

28. Operación diaria

```
cd deploy
docker compose up -d           # levantar
docker compose --profile tools down # bajar (NUNCA con -v: borra la base)
bash scripts/lab-001-completo.sh # repetir todos los laboratorios
```

- **Reconstruir el teléfono simulado:** `./gradlew :wallet-sim:installDist && docker compose --profile tools build holder`.
- **Subir el nivel mínimo exigido:** `WALLET_MIN_LEVEL=TEE` en `deploy/.env` y reiniciar `wallet`.

ANEXOS

Anexo A — Los scripts

Script	Para qué sirve
scripts/lab-001-completo.sh	Ejecuta los laboratorios A a E en una sola sesión
scripts/e2e.sh	Recorrido de extremo a extremo (121 comprobaciones)
scripts/gen-dev-certs.sh	Certificados de laboratorio
scripts/generar-pdf.sh	Convierte un informe a PDF

Anexo B — Autocomprobación (con respuestas)

1. ¿Qué operación del contrato usa la clave privada? → Solo `sign`.
2. ¿Qué niveles de protección existen? → `SOFTWARE`, `TEE`, `STRONGBOX`.
3. ¿Qué prueba que una attestation no es vieja? → El desafío incrustado coincide con el emitido.
4. ¿Qué ocurre si el origen de la clave no es `GENERATED`? → `KEY_NOT_GENERATED_IN_HARDWARE`.
5. ¿Cuánto dura el token de recuperación? → 10 minutos y un solo uso.
6. ¿Cómo se guarda el código de recuperación? → Resumen PBKDF2 con sal.
7. ¿Qué pasa con la cartera anterior tras recuperar? → Se revoca.
8. ¿Por qué hay que reemitir credenciales? → Estaban ligadas a la clave anterior.
9. ¿Qué garantiza «una cartera activa por ciudadano»? → Un índice único parcial en la base.
10. ¿Qué NO se probó? → Un chip real y cadenas de attestation reales.

Anexo C — Ejercicios del método (con respuesta modelo)

Ejercicio 1. Descompón el criterio 3. → Verbo: *coincide*. Evidencia: ficha técnica y verificación. Casos negativos: exagerar, declarar de menos, raíz falsa, attestation vieja, clave importada, arranque no verificado. Mecanismo: `KeyAttestationVerifier` + comparación estricta.

Ejercicio 2. Diseña una prueba para «el servidor no accede a la clave privada». → Registrar todas las formas textuales de las claves privadas de prueba, volcar la base completa y buscarlas; incluir un control positivo para descartar una búsqueda vacía (§17.4).

Ejercicio 3. ¿Qué pruebas harías con un teléfono Android real? → Generar la clave con Keystore/StrongBox; leer `KeyInfo.isInsideSecureHardware`; comprobar `PrivateKey.encoded == null`; intentar exportar y esperar fallo; obtener la cadena real y verificarla contra las raíces de Google; comparar nivel declarado y verificado.

Anexo D — Referencias

- Android Keystore y Key Attestation (`KeyDescription` , OID `1.3.6.1.4.1.11129.2.1.17`); StrongBox.
- iOS Secure Enclave; WebAuthn / FIDO2.
- W3C Verifiable Credentials Data Model (rol del *holder*).
- RFC 7638 (huella de clave), RFC 8018 (PBKDF2).
- `docs/ANDROID-REFERENCIA.md` , `docs/ANALISIS-BASES-EUDI.md` , `docs/VERSIONES-NORMATIVAS.md` , `docs/MARCO-CONCEPTUAL.md` .